

中山大學
SUN YAT-SEN UNIVERSITY

PersonalTax

面向对象程序设计文档

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 李文军

起 始 日 期: 2026 年 3 月

结 束 日 期: 2026 年 3 月

实 验 地 点: B201

目录

1 需求分析与对问题的理解 **2**

2 面向对象建模与架构设计 **2**

2.1 实验结构说明 2
2.2 架构图设计 2
2.3 类图设计 3
2.4 算法活动图 5
2.5 系统计税交互时序图 5

3 算法与设计分析 **6**

3.1 摒弃 `if-else` 的超额累进算法 6
3.2 扩展性优势 6
3.3 程序的健壮性 7
3.4 自动化回归测试 (Regression Testing) 7

1 需求分析与对问题的理解

本实验的核心目标是设计一个基于命令行的个人所得税计算系统。根据我国税法规定，应缴纳的个人所得税依赖于“应纳税所得额”（即总收入减去免征基数/起征点），并按照超额累进税率表进行分段计算。

我认为本系统在设计时面临的核心痛点在于**政策的易变性**：

1. **起征点变化**：根据课程文档，历史上的起征点曾经历过 800 元、1600 元、3500 元乃至 5000 元的多次调整。
2. **税率分级变化**：税率的级数和对应的百分比也可能随立法修订而改变（例如从 9 级调整为 7 级）。

因此，如果采用传统的结构化程序设计，将所有的税率区间和百分比通过冗长的 `if-else` 语句硬编码（Hardcode）在函数内部，未来一旦政策变动，代码的修改成本将极高，且极易引入错误。

本系统遵循**面向对象设计原则**，实现数据与逻辑的解耦，以保证程序具备良好的**可复用性**与**可扩展性**。

2 面向对象建模与架构设计

2.1 实验结构说明

本实验的项目目录结构进行了明确的模块化划分，将源代码、说明文档、自动化脚本与测试用例进行了解耦。具体的物理目录结构如下：

学号姓名/（根文件夹）

```
src/                # 存放所有 .java 源代码文件
    TaxApp.java
    TaxCalculator.java
    TaxRateItem.java

doc/                # 存放 javadoc 生成的 HTML 说明文档（index.html 等）

scripts/            # 存放编译与运行脚本
    build.bat       # 编译脚本
    run.bat         # 运行脚本
    regression.bat  # 回归测试脚本（如有）

test_cases/         # 存放测试用例相关文件（输入数据、预期结果等）

design.pdf          # 面向对象设计文档（包含 UML 类图等）
readme.txt          # 个人信息及实验体会
```

2.2 架构图设计

在宏观上采用了清晰的三层架构设计，系统包图与组件之间的单向依赖关系如图 1 所示。

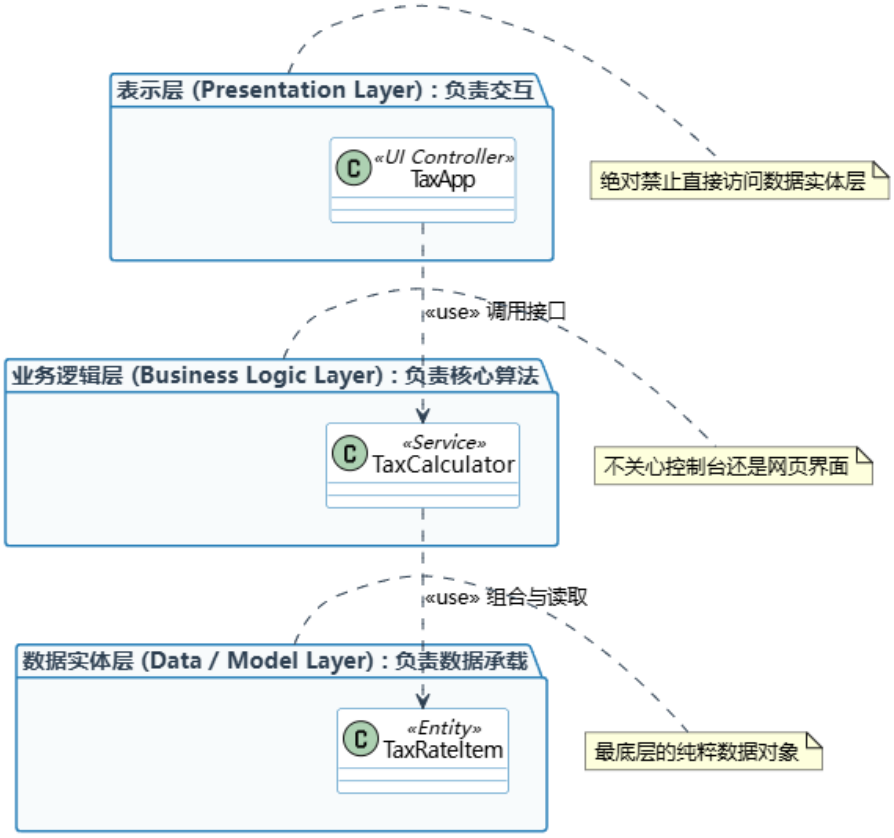


图 1: 个人所得税计算系统包图 / 三层架构图

- **三个层级包**：自上而下分别定义了三个模块：
  - **表示层 (Presentation Layer)**：内有 `TaxApp`，负责交互。
  - **业务逻辑层 (Business Logic Layer)**：内有 `TaxCalculator`，负责核心算法。
  - **数据实体层 (Data / Model Layer)**：内有 `TaxRateItem`，负责数据承载。
- **依赖连线**：
  - 从 `TaxApp` 指向 `TaxCalculator` 的虚线箭头标注了 `<<use>>` 调用接口，说明 UI 层依赖于逻辑层。
  - 从 `TaxCalculator` 指向 `TaxRateItem` 的虚线箭头标注了 `<<use>>` 组合与读取，说明逻辑层需要获取底层数据。

2.3 类图设计

本项目类图如图 2 所示，详细描绘了类及属性方法：

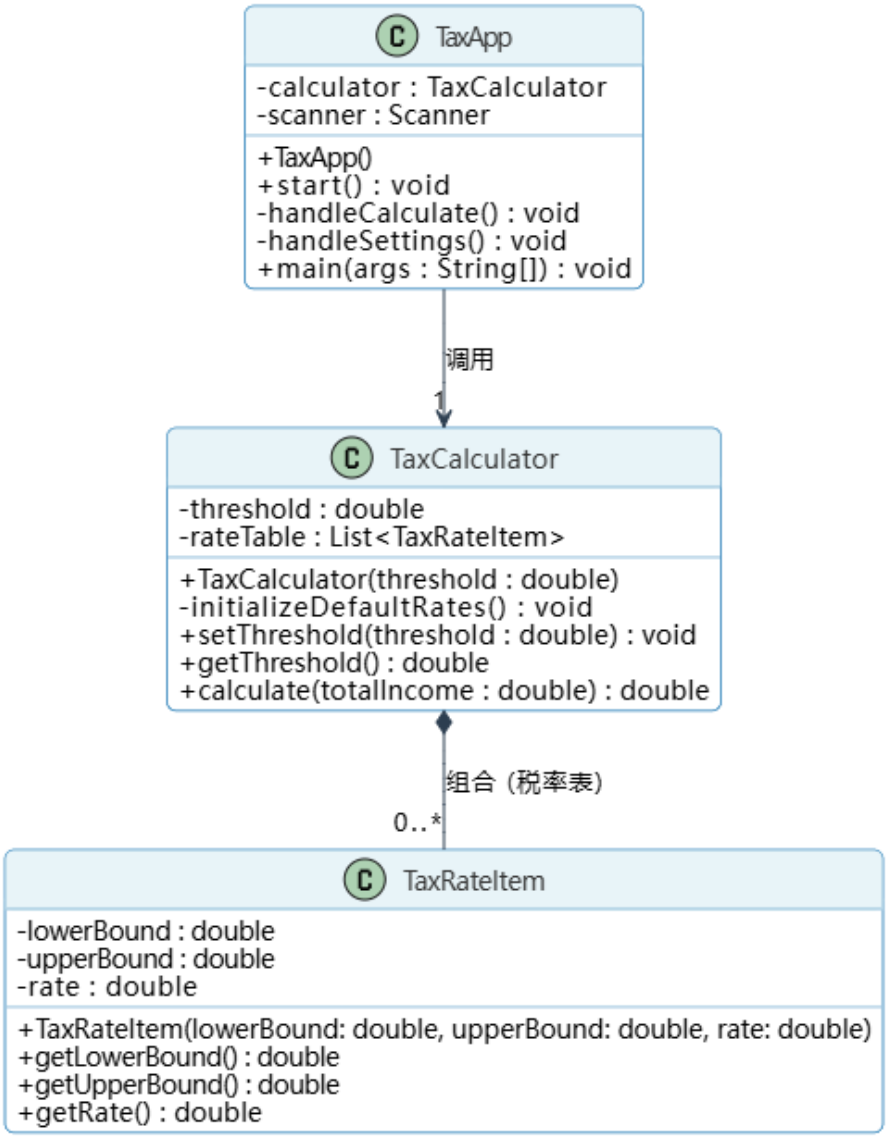


图 2: 核心组件 UML 类图

- `TaxApp` 类：
  - **私有属性（减号 - 表示）**：包含 `-calculator : TaxCalculator` 与 `-scanner : Scanner`，将外部依赖安全隐藏。
  - **公有/私有方法（加号 + / 减号 - 表示）**：包含了构造函数 `+TaxApp()`、启动方法 `+start() : void`、主入口 `+main(args : String[]) : void`，以及两个被封装在内部不对外暴露的私有处理方法 `-handleCalculate() : void` 和 `-handleSettings() : void`。
  - **依赖关系**：它通过一条标注为“**调用**”的带箭头实线指向 `TaxCalculator`，箭头旁的数字“**1**”代表一个 App 实例仅关联一个计算器引擎。
- `TaxCalculator` 类：
  - **属性与方法**：如图，不再赘述。
  - **组合关系**：它通过一条起点为实心黑菱形的实线连接至 `TaxRateItem`。这个**实心菱形代表组合（Composition）**关系，线上的文字“组合 (税率表)”及数字“**0..***”（代表 0 到

多个）精确说明了计算器是由多条税率项构成的一个有机整体。

- **`TaxRateItem` 类：**
  - **属性与方法：**如图，不再赘述。
  - 仅暴露了对应的公有（+）Getter 方法，确保了数据的只读安全性。

2.4 算法活动图

使用活动图语法，详细刻画了 `TaxCalculator.calculate()` 方法内部的指令流转过程，如图 3 所示。

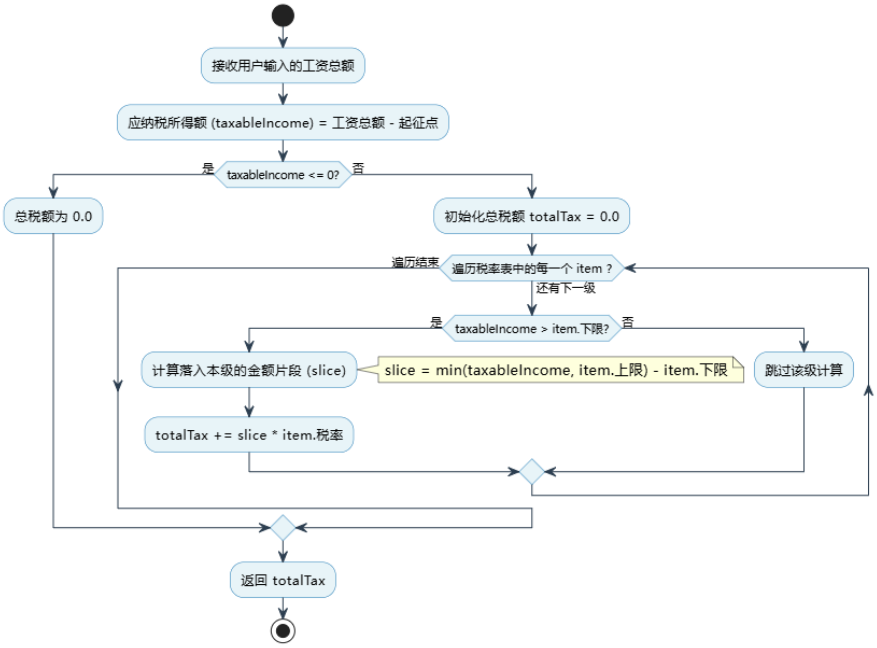


图 3: 超额累进算法活动图

1. 首先计算应纳税所得额（总收入减去起征点）。如果结果小于等于 0，则直接返回 0 税额。
2. 否则，进入循环遍历税率表的每一项。对于每个税率项，判断应纳税所得额是否超过该档次的下限。
3. 如果超过，则计算该档次内的金额切片（即应纳税所得额与该档次上限的较小值减去下限），并乘以对应的税率累加到总税额中。
4. 循环结束后返回最终计算出的总税额。

2.5 系统计税交互时序图

时序图展现了系统在时间维度上的动态消息调用顺序，揭示了多个组件间是如何协同完成一次核心计税的，如图 4 所示。

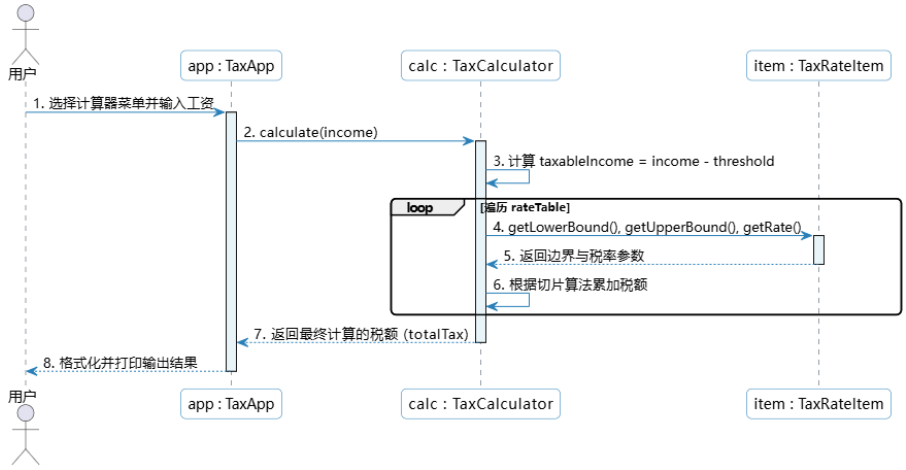


图 4: 系统计税交互时序图

如图所示，用户输入总收入后，`TaxApp` 通过调用 `TaxCalculator.calculate()` 方法，将输入数据传递给业务逻辑层。计算器内部根据税率表进行分段计算，并将结果返回给 `TaxApp`，最终由 UI 层负责输出结果给用户。

3 算法与设计分析

3.1 摒弃 `if-else` 的超额累进算法

在 `TaxCalculator` 的核心方法 `calculate` 中，摒弃了传统的阶梯式 `if-else` 或 `switch-case` 判定结构，转而采用遍历税率表对象的切片累加算法:

```
public double calculate(double totalIncome) {
    double taxableIncome = totalIncome - threshold;
    if (taxableIncome <= 0) return 0.0;

    double totalTax = 0.0;
    for (TaxRateItem item : rateTable) {
        if (taxableIncome > item.getLowerBound()) {
            // 计算落在该档次内的金额切片
            double slice = Math.min(taxableIncome, item.getUpperBound()) - item.
                getLowerBound();
            totalTax += slice * item.getRate();
        }
    }
    return totalTax;
}
```

Listing 1: 核心切片累加算法

3.2 扩展性优势

得益于上述面向对象的数据抽象与循环遍历算法，如果未来个税政策发生根本性改变（如增设超高收入人群的 6 级、7 级税率），**核心算法代码 (`calculate` 方法) 无需进行任何修改**。开发者仅

需在系统初始化阶段，向 rateTable 列表中动态 add() 新的 TaxRateItem 实例即可。这深刻体现了面向对象建模在应对复杂多变业务时的扩展性。

3.3 程序的健壮性

在 TaxApp 的交互层，广泛使用了 try-catch 异常捕获机制。当用户在要求输入金额的环节输入非法字符（如字母或特殊符号）时，程序能够拦截 NumberFormatException，并友好地提示“输入数据格式不正确”，随后安全返回主菜单。这极大增强了系统的健壮性。

3.4 自动化回归测试 (Regression Testing)

为了验证系统的准确性并防止未来的修改引入新漏洞，系统构建了自动化回归测试模块：

1. **测试用例库**：在 test_cases 目录下，设计了包含普通收入用例、临界点边界值用例（如收入恰好等于起征点）、极高收入用例以及非法输入用例的结构化测试集。
2. **批处理脚本集成**：编写了 regression_test.bat 脚本。当调整了起征点逻辑或新增税率后，一键运行该脚本，即可通过 I/O 重定向将测试集的数据批量输入程序，并将结果汇总至 test_report.log 中。。